Binary Heaps

Heaps

AVL and other balanced search trees give us $O(\log n)$ worst case performance for all operations. Hash tables give improved best and average case performance of $O(1)$, but sacrifice ordering and worst case performance.

What if we still want ordered data, but don't need all of the basic operations insert, delete and search.

You should have previously seen the concept of a priority queue. Order is important in priority queue because we want quick access to the highest priority item (a hash table would need to search all items for this), but we don't need a general search or delete operation. The only operations we need are insert and delete/examine the highest priority item.

Heaps provide a simple and efficient implementation of a priotity queue.

Content

- Heaps
- Insertion
- Deletion
- Heapify

Heaps

The basic operations of a priority queue are inserting an item and deleting/examining the item with minimum priority (or maximum priority depending on your definition).

A priority queue could be implemented with an ordered array. This would make it easy to find the top priority item but, as we have seen, insert would be $O(n)$.

We could use an unordered array. This would give $O(1)$ for inserting, but finding the top priority item would be $O(n)$. The same would be true for linked lists.

We could just use an AVL tree, which would give $O(\log n)$ for insert and delete, which is pretty good. But even though AVL trees are good in $O(\cdot)$ terms, they are quite complex to implement with fairly poor constant factors involved.

Heaps provide the operation we need in $O(\log n)$, but with much better constant factors, so are quite a bit faster than AVL trees in practice.

Heap Structure

A heap is *conceptually a complete* binary tree, i.e. it is completely filled from the top to bottom. This structure ensures that the height is always $O(\log_2 n)$, so all operations that are $O(height)$ are $O(\log_2 n)$.

Since a heap is a binary tree, it could be implemented using a linked BST-style structure. But there's a reason that I used the word *conceptually* above - a heap is almost always implemented as an array. This is possible because of the *highly constrained* structure of the tree - we know where every node needs to go in the array.

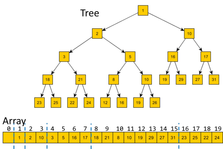
Typically we start filling the array at index 1, leaving index 0 empty. This is not strictly required, but makes some of the maths a bit nicer.

We simply fill the array starting at index 1 for the root, and indices 2 and 3 for the next layer, indices 4-7 for the next layer and so on.

This leads to some simple math for traversing the tree:

- The parent of a node at index i is at index i/2
- The left child of a node at index i is at index 2*i
- The right child of a node at index i is at index 2*i + 1

In the following diagram the conceptual tree at the top is implemented as the array below.



First note that the tree is complete - every layer is filled down to the last layer which is filled left-to-right. Applying the rules above we see that the node containing 16 (at index 6) has it's parent at index 3 and it's children at indices 12 and 13.

When conceptualising a heap, we usually draw it as a tree. We really only think of the array for implementation in code.

Heap Order

Each item in a (min)heap is smaller than all of it's decendants. For example, the root of a (min)heap is the smallest value in the entire heap and the '5' in the middle of the heap above is smaller than all of the values in the branch that it is the root of.

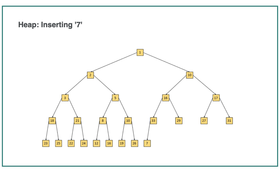
This means that for the delete operation, it's easy to find the item to be deleted - it's always at the root of the tree (or the first item in the array).

Heap Insertion

To insert an item into a heap, we start by placing it at the next available position at the bottom of the heap. This will be the next position in the array.

We then bubble (or percolate, according to some) the item up to satisfy the order property. Basically we just compare the item with it's parent, and if it's smaller (for a min heap), the item and parent swap positions. And repeat this until the item is larger than it's parent, or at the root.

At most the item moves from the bottom of the tree to the top. And since the height of the tree is $O(\log n)$, insert must also be $O(\log n)$. Importantly, the $O(\log n)$ operations are just swapping numbers in an array - i.e. very fast!



And here's a larger example:

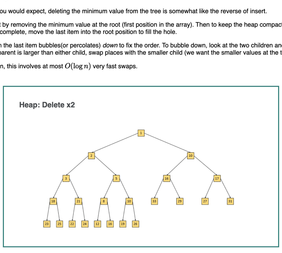
Heap Deletion

As you would expect, deleting the minimum value from the tree is somewhat like the reverse of insert.

Start by removing the minimum value at the root (first position in the array). Then to keep the heap compact and complete, move the last item into the root position to fill the hole.

Then the last item bubbles(or percolates) down to fix the order. To bubble down, look at the two children and if the parent is larger than either child, swap places with the smaller child (we want the smaller values at the top).

Again, this involves at most $O(\log n)$ very fast swaps.



And here's another example:

Building an Entire Heap

When you are given items to insert into a heap one at a time, the insert operation is very fast. But sometimes you are given a bunch or data all at once that you need to turn into a heap - e.g. an already filled, randomly sorted array.

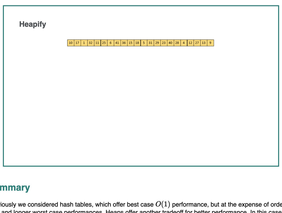
It turns out that you can turn this into a heap even faster than inserting the items one at a time. This operation is often referred to as *heapify*.

Inserting n items one at time would take $O(n \log n)$ time, but heapify can achieve this in just $O(n)$, i.e. the same amount of time as it would take to just look at the n items!

We start by simply treating the randomly sorted array as a binary tree - obviously, the order will be wrong at this point.

Then, starting at the bottom layer of the tree, we bubble items down, if they need to, to fix the order. Then move up to the next layer and repeat until we get to the root.

At first glance this looks like it should still be $O(n \log n)$, since every item needs to bubble to it's correct position. The reason why it turns out to be faster is that most of the items in a heap are near the bottom, and only a few are near the top. Since we are only ever bubbling down, most items don't have to go very far, only a few need to move the $O(\log n)$ height of the tree. See "Time Complexity of building a heap" for a proof.



Summary

Previously we considered hash tables, which offer best case $O(1)$ performance, but at the expense of ordered data and longer worst case performances. Heaps offer another tradeoff for better performance. In this case asymptotic performance is still $O(\log n)$, but with good constant factors. However, heaps only provide priority queue operations.

We have only considered a very basic binary heap. There are many other variations of the heap concept that have different trade-offs, which you can read about below.

Further Reading

Different types of heaps:

- *Binomial heaps provide very fast insert $O(1)$, but also are faster for merging two heaps ($O(logn)$ vs $O(n)$ for binary heaps).*
- *Fibonacci heaps also provide $O(1)$ for inserting, but also for merging heaps. Importantly, reducing the key/priority of a node is also $O(1)$ - this is very important for Dijkstra's algorithm (a shortest path algorithm that we will discuss later).*
- *Wikipedia has a list of even more heaps with their performance for different operations.*

Visualgo has a good visualisation for understanding binary heap operations.